

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №5.

Тема: «Наследование. Виртуальные функции. Полиморфизм»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 10

Выполнила: студентка группы РИС-25-26

Абрамова Валерия Андреевна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи

Поля:

- Первое число (first) – int
- Второе число (second) – int
- Третье число (third) – int

Методы:

- изменение полей
- увеличение каждого поля на 1

Производный класс: DATE (дата)

Поля: год, месяц, число

Задания:

- переопределить методы увеличения полей на 1 (с учётом календаря)
- определить метод увеличения даты на n дней

Анализ задачи

1. Абстрактный класс `Triad` хранит три целых числа и определяет чисто виртуальные методы `increment` (увеличение полей на 1) и `print` (вывод), задавая интерфейс для всех производных классов.
2. Производный класс `Date` наследует `Triad`, добавляет поля год, месяц, день, переопределяет `increment` для увеличения даты на 1 день и `print` для вывода даты, а также добавляет метод `addDays` для увеличения даты на n дней и вспомогательный метод `normalize` для приведения даты к корректному виду.
3. Класс `Vector` хранит динамический массив указателей на `Triad`, предоставляет метод `add` для добавления объектов, метод `incrementAll` для вызова `increment` у всех элементов и перегруженный оператор вывода.
4. Взаимодействие классов заключается в том, что `Vector` хранит указатели на базовый класс `Triad`, но реально в него добавляются объекты производного класса `Date`. При вызове `incrementAll` через указатель на `Triad` вызывается переопределённый метод `Date`, что демонстрирует полиморфизм. Оператор вывода `Vector` вызывает виртуальный метод `print` каждого объекта, который у `Date` выводит дату. Назначение: `Triad` нужен для определения общего интерфейса, `Date` — для реализации конкретной логики дат, `Vector` — для хранения объектов иерархии и демонстрации полиморфного поведения.

Код

```
#include <iostream>
#include <locale>
using namespace std;

class Triad {
protected:
```

```

    int first;
    int second;
    int third;

public:
    Triad(int f = 0, int s = 0, int t = 0) : first(f), second(s), third(t) {}

    virtual void setFirst(int f) { first = f; }
    virtual void setSecond(int s) { second = s; }
    virtual void setThird(int t) { third = t; }

    virtual void increment() = 0;
    virtual void print(ostream& out) const = 0;

    virtual ~Triad() {}
};

class Date : public Triad {
private:
    int year;
    int month;
    int day;

    bool isLeapYear(int y) const {
        return (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
    }

    int daysInMonth(int m, int y) const {
        switch (m) {
            case 1: case 3: case 5: case 7: case 8: case 10: case 12:
                return 31;
            case 4: case 6: case 9: case 11:
                return 30;
            case 2:
                return isLeapYear(y) ? 29 : 28;
            default:
                return 30;
        }
    }

    void normalize() {
        while (day > daysInMonth(month, year)) {
            day -= daysInMonth(month, year);
            month++;
            if (month > 12) {
                month = 1;
                year++;
            }
        }
        while (day < 1) {
            month--;
            if (month < 1) {
                month = 12;
                year--;
            }
            day += daysInMonth(month, year);
        }
    }

public:
    Date(int d = 1, int m = 1, int y = 2000, int f = 0, int s = 0, int t = 0)
        : Triad(f, s, t), day(d), month(m), year(y) {}

    void increment() override {
        day++;
        normalize();
    }
}

```

```

    void addDays(int n) {
        day += n;
        normalize();
    }

    void print(ostream& out) const override {
        out << day << "." << month << "." << year;
    }
};

class Vector {
private:
    Triad** elements;
    int size;
    int capacity;

public:
    Vector() : elements(nullptr), size(0), capacity(0) {}

    void add(Triad* obj) {
        if (size >= capacity) {
            capacity = (capacity == 0) ? 2 : capacity * 2;
            Triad** newElements = new Triad * [capacity];

            for (int i = 0; i < size; i++) {
                newElements[i] = elements[i];
            }

            delete[] elements;
            elements = newElements;
        }
        elements[size++] = obj;
    }

    friend ostream& operator<<(ostream& out, const Vector& vec) {
        for (int i = 0; i < vec.size; i++) {
            out << "Элемент " << i + 1 << ": ";
            vec.elements[i]->print(out);
            out << endl;
        }
        return out;
    }

    void incrementAll() {
        for (int i = 0; i < size; i++) {
            elements[i]->increment();
        }
    }

    ~Vector() {
        for (int i = 0; i < size; i++) {
            delete elements[i];
        }
        delete[] elements;
    }
};

int main() {
    setlocale(LC_ALL, "Russian");

    Vector vec;

    vec.add(new Date(28, 2, 2020));
    vec.add(new Date(31, 12, 2023));
    vec.add(new Date(1, 1, 2024));

    cout << " Исходные даты " << endl;
    cout << vec;
}

```

```

vec.incrementAll();

cout << "\n После увеличения всех дат на 1 день " << endl;
cout << vec;

Triad* ptr = new Date(30, 12, 2023);
cout << "\ Работа с отдельным объектом " << endl;
cout << "Исходная дата: ";
ptr->print(cout);
cout << endl;

ptr->increment();
cout << "После increment: ";
ptr->print(cout);
cout << endl;

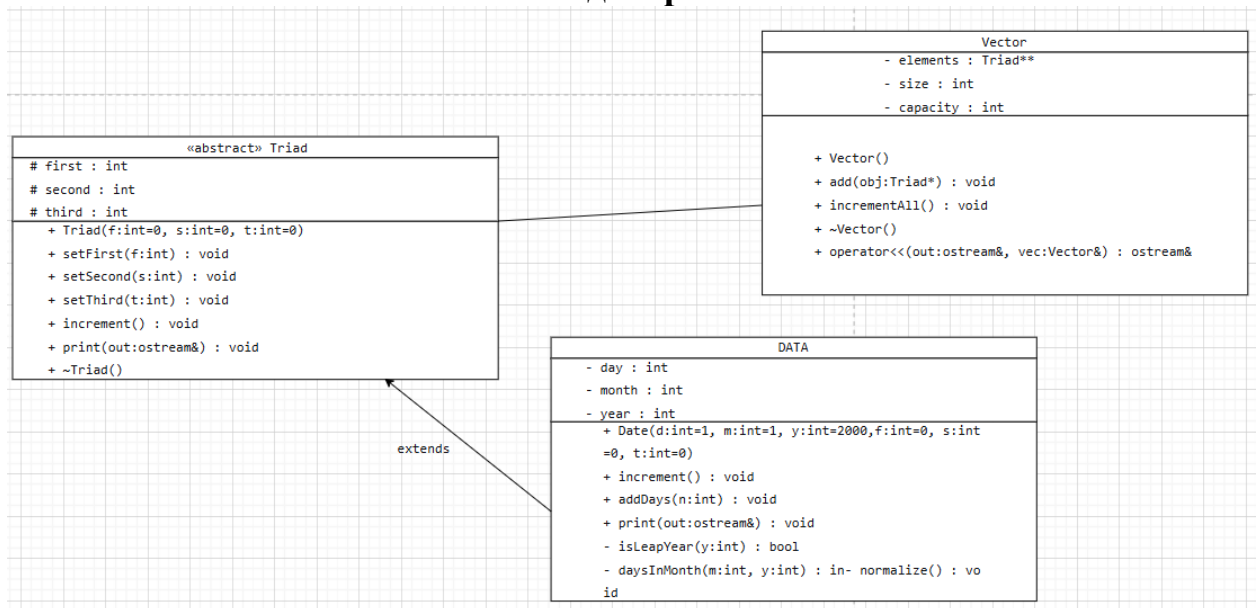
Date* datePtr = dynamic_cast<Date*>(ptr);
if (datePtr) {
    datePtr->addDays(10);
    cout << "После addDays(10): ";
    datePtr->print(cout);
    cout << endl;
}

delete ptr;

return 0;
}

```

UML диаграмма



Вывод программы

```
Исходные даты
Элемент 1: 28.2.2020
Элемент 2: 31.12.2023
Элемент 3: 1.1.2024

После увеличения всех дат на 1 день
Элемент 1: 29.2.2020
Элемент 2: 1.1.2024
Элемент 3: 2.1.2024

Работа с отдельным объектом
Исходная дата: 30.12.2023
После increment: 31.12.2023
После addDays(10): 10.1.2024
```

Ответы на вопросы

1. Чисто виртуальный метод в C++ объявляется с помощью `= 0`. Он не имеет реализации в базовом классе. Отличие от виртуального метода: виртуальный метод имеет реализацию в базовом классе и может быть переопределён, а чисто виртуальный обязан быть переопределён в производном классе.
2. Абстрактный класс в C++ — это класс, содержащий хотя бы один чисто виртуальный метод. Объект такого класса создать нельзя.
3. Абстрактные классы предназначены для задания интерфейса. Они определяют, какие методы должны реализовать производные классы, но не предоставляют реализацию этих методов.
4. Полиморфные функции в C++ — это виртуальные функции, поведение которых зависит от фактического типа объекта. Вызов такой функции через указатель или ссылку на базовый класс приводит к выполнению версии из производного класса.
5. Принцип подстановки — это правило, по которому объект производного класса можно использовать вместо объекта базового класса. Полиморфизм — это механизм, который позволяет этому правилу работать динамически, то есть выбирать нужную реализацию метода во время выполнения.
6. Пример иерархии с абстрактным классом в C++:

```
class Фигура {
public:
    virtual double площадь() = 0;
};
```

```
class Круг : public Фигура {
```

```
public:  
    double площадь() override { return 3.14 * r * r; }  
};
```

```
class Прямоугольник : public Фигура {  
public:  
    double площадь() override { return a * b; }  
};
```

7. Пример полиморфной функции в C++:

```
class Животное {  
public:  
    virtual void голос() { }  
};  
  
class Собака : public Животное {  
public:  
    void голос() override { }  
};  
  
void издатьЗвук(Животное& a) {  
    a.голос();  
}
```

Функция `голос()` ведёт себя по-разному в зависимости от того, объект какого типа передан в `издатьЗвук`.

8. Механизм позднего связывания в C++ используется при вызове виртуальных функций через указатель или ссылку на базовый класс, когда тип объекта неизвестен на этапе компиляции и определяется во время выполнения программы.